

Plan de trabajo en equipo — Parcial-1

Proyecto: Gestión de Inventario (MVC + Patrones + Testing)
Integrantes: Aguirre Claudio · Albano Julieta · Windholz Cristhian
Basado en: auditoría técnica y división de tareas (sin modificar código en este documento)

Este documento reparte el trabajo pendiente para cumplir la consigna: MVC estricto, tests unitarios, README con diagrama, y evidencia de flujo Git en ramas.

Visión general

Integrante	Foco principal	Rama sugerida
Aguirre Claudio	Dominio + MVC estricto (backend)	rama-AguirreClaudio
Albano Julieta	Testing + calidad de lógica testeable	rama-AlbanoJulieta
Windholz Cristhian	README, diagrama, Git/documentación + pulido frontend	rama-WindholzCristhian

Cada integrante debe **commitear en su rama** y luego hacer **merge** a una rama común (`main` o `develop`) con mensajes claros.

1. Aguirre Claudio — Backend y MVC estricto

Objetivo: que la lógica de negocio no quede “escondida” en el controlador HTTP.

#	Tarea	Entregable	Criterio de “listo”
1.1	Crear capa de dominio/servicio (ej. <code>ProductoService</code> o ampliar modelo) que use Strategy y Decorator	Archivo(s) en <code>src/</code>	<code>ProductoController</code> solo orquesta HTTP y llama al servicio
1.2	Mover desde <code>ProductoController</code> el <code>map</code> con <code>decorarProducto</code> + <code>calcularPrecio</code> al servicio/modelo	PR en su rama	Controlador sin reglas de precio ni armado de descripción

#	Tarea	Entregable	Criterio de “listo”
1.3	Dejar <code>ProductoModel</code> solo en persistencia (CRUD) y <code>Producto</code> como entidad	Código documentado en README (1 párrafo)	Separación clara Modelo datos vs lógica
1.4	Revisar Singleton : comentar trade-off conexión única vs pool	Nota en README sección Singleton	Pueden explicarlo en defensa oral
1.5	Validaciones de entrada (crear/actualizar: campos obligatorios, precio ≥ 0) en capa correcta	Validación en backend	POST/PUT rechazan datos inválidos con 400

Commits sugeridos:

- refactor: mover Strategy y Decorator a capa de dominio
- feat: validaciones de producto en servicio

Dependencia: Albano necesita que Strategy/Decorator sean importables y testeables sin Express (idealmente desde el servicio).

2. Albano Julieta — Testing (requisito bloqueante)

Objetivo: 3–4 tests unitarios automatizados, independientes de MySQL.

#	Tarea	Entregable	Criterio de “listo”
2.1	Elegir runner (Jest o <code>node:test</code>) y configurar <code>package.json</code>	<code>devDependency</code> + script	<code>npm test</code> termina en verde
2.2	Carpeta <code>tests/</code> o <code>__tests__/</code> con convención clara	Estructura en repo	Profesor encuentra tests al instante
2.3	Test 1: <code>PrecioListaStrategy</code> — precio sin cambio	1 assert	Independiente de DB
2.4	Test 2: <code>PrecioMayoristaStrategy</code> — 15% descuento (ej. 100 → 85)	1 assert	Mismo criterio
2.5	Test 3: <code>PrecioPromocionStrategy</code> — stock ≥ 10 vs < 10	2 casos o 2 tests	Cubre rama del <code>if</code>
2.6	Test 4 (recomendado): <code>AlertaStockBajoDecorator</code> o	String con etiqueta esperada	Sin fetch, sin MySQL

#	Tarea	Entregable	Criterio de "listo"
	EtiquetaMarcaPremiumDecorator con mock		
2.7	Si testean ProductoModel : mockear Singleton o no testear SQL	Documentar en README	Tests no requieren XAMPP

Commits sugeridos:

- test: agregar Jest y tests de PrecioStrategy
- test: decorador stock bajo y marca premium

Dependencia: idealmente después del refactor de Claudio; si no, testear PrecioStrategy.js y ProductoDecorator.js directamente.

Coordinación con Claudio: acordar si los tests importan desde src/patterns/... o desde el nuevo servicio.

3. Windholz Cristhian — README, diagrama, Git y frontend

Objetivo: cumplir entregables documentales, evidencia de trabajo en equipo y pulir MVC en cliente.

#	Tarea	Entregable	Criterio de "listo"
3.1	Diagrama en README (Mermaid): navegador → API → Controller → Servicio/Modelo → Singleton → MySQL	Bloque en README	Cumple ítem "diagrama" de la consigna
3.2	Ampliar justificación de patrones (problema real + alternativa descartada)	Sección Patrones	Defendible en oral
3.3	Sección "Ejecutar tests" (npm test) cuando Albano termine	README actualizado	Guía install + run + test
3.4	Sección Git / trabajo en equipo: ramas, quién hizo qué, cómo mergear	Tabla integrante ↔ commits	Evidencia 50/50
3.5	README.Md → README.md si el repo lo exige	Consistencia GitHub	Visible en remoto
3.6	Frontend MVC: error en ProductoUIController que escribe innerHTML → delegar a ProductoView	1 método en vista	Vista única dueña del DOM
3.7	Comitear lo untracked (src/ , public/js/ , server.js , etc.)	Historial con ≥3 autores/commits	No solo first commit

#	Tarea	Entregable	Criterio de "listo"
3.8	Guion de defensa oral (respuestas a preguntas incómodas)	Doc interno o apéndice README	Los 3 saben responder

Commits sugeridos:

- docs: diagrama MVC y justificación de patrones
- fix: errores de lista delegados a ProductView
- chore: consolidar estructura public/ y server.js

Dependencia: diagrama de tests después de Albano; diagrama de arquitectura puede hacerse ya (anotar "Servicio" como pendiente de Claudio).

Orden sugerido (cronograma mínimo)

Días 1–2 (paralelo)

- **Claudio:** refactor dominio en backend
- **Albano:** tests sobre PrecioStrategy / Decorator
- **Cristhian:** diagrama + commits iniciales + README base

Día 3

- Claudio termina servicio; Albano integra script npm test ; Cristhian actualiza README

Día 4

- Merge de las 3 ramas; resolver conflictos en ProductoController / package.json

Día 5

- Prueba manual CRUD + npm test ; repaso oral los 3 juntos

Flujo resumido: Semana 1 (paralelo: dominio, tests, docs) → Semana 2 (integración: merge, ajuste tests, README final, prueba npm start + npm test).

Reparto equilibrado (~33% c/u)

Integrante	Esfuerzo estimado	% del trabajo faltante
Claudio	Refactor medio-alto (arquitectura)	~35%

Integrante	Esfuerzo estimado	% del trabajo faltante
Albano	Tests + config (obligatorio)	~30%
Cristhian	Docs + Git + fix frontend + coordinación	~35%

Checklist final del equipo (antes de entregar)

- ☐ `npm start` levanta app y CRUD funciona
- ☐ `npm test` → 3–4 tests en verde, sin MySQL
- ☐ Controlador backend sin lógica de precio/decoración
- ☐ README con diagrama, patrones, ejecución y tests
- ☐ Historial Git con commits de los 3 en ramas distintas + merge
- ☐ Los 3 pueden responder: dónde está el Modelo, qué testean y por qué Decorator

Qué NO hacer (para no pisarse)

Evitar	Quién lo evita
Dos personas editando <code>ProductoController</code> a la vez	Claudio dueño del backend; Albano no lo toca salvo merge
Albano y Claudio cambiando <code>package.json</code> sin hablar	Albano pide dependencia de test; Claudio no agrega otras sin avisar
Cristhian documenta lógica que aún no existe en código	Esperar merge de Claudio para diagrama final